

Introduction

This specification is about a framework to establish communication channels between tasks, via brokers:

- Each task is a thread
- Each broker is uniquely named
- Channels are established via an accept/connect rendez-vous.
- Channels are byte streams
 - point-to-point
 - full-duplex
 - FIFO and lossless.

The interfaces are the following:

```
package edu.polytech.channels;
public interface Broker {
    String getName();
    Channel connect(String name, int port);
    Channel accept(int port);
}
```

```
package edu.polytech.channels;
public interface Channel {
    int read(byte bytes[], int offset, int length);
    int write(byte bytes[], int offset, int length);
    boolean disconnected();
    void disconnect();
}
```

We need an execution model, either thread-oriented or event-oriented. We will choose a thread-oriented model here, so we need the concept of tasks, each task is essentially a thread associated with one broker. As a first approximation, a task can be defined as the following interface:

```
package edu.polytech.channels;
public class Task {
    public static Task task();
    public Broker getBroker();
    public Broker newBroker(String name);
    public Task newTask(Broker b, Runnable r, String name);
}
```

Additionally, one needs to be able to bootstrap a runtime, creating the first brokers and the first tasks.

```
package edu.polytech.channels;
public interface Bootstrap {
    public Broker newBroker(String name);
    public Task newTask(Broker b, Runnable r, String name);
}
```

Each implementation will offer a "bootstrap class", implementing the above interface, and defining a public constructor without arguments.

- This class must be instantiated once and the instance must be used to create the first brokers and the first tasks.
- Once the runtime is bootstrapped, the instance should no longer be used, one should use the methods on the class Task to create new brokers and new tasks.

Overview: Broker / Channel

Brokers are named intermediaries that enable tasks to establish channels. Each broker has a unique name. Each channel is a communication channel between two end points, that is, a point-to-point stream of bytes. A channel is full-duplex, that is, each end point can be used to read or write bytes. A channel, obtained from a rendez-vous between a paired 'accept' or 'connect' operation. Once created, a channel is fully connected. A connected channel is FIFO and lossless. A connected channel can be disconnected at any time, by either sides.

The typical use of a channel is by two tasks to establish a full-duplex communication. However, there is no ownership between channel and tasks, any task may read or write in any channel it has a reference to. The following rules apply though:

- It is thread-safe for two tasks communicating through a channel to read or write concurrently, each at their respective end point of the channel.
- Locally, at one end point, two tasks, one reading and the other writing, operating concurrently is also safe.

Important:

- a blocked read operation does not prevent write operations to complete
- a blocked write operation does not prevent read operations to complete
- Concurrent read operations or concurrent write operations, on the same end point, by different tasks, are not safe since they do not have a clear operational semantic given the byte-stream nature of channels.

Connecting

A channel is established, in a fully connected state, when a request to 'connect' matches a request to 'accept'. When connecting, the given name is the one of the remote broker, the given port is the one of an

accept on that remote broker.

There is no precedence between connect and accept, this is a symmetrical rendez-vous: the first operation waits for the second one. Both operations are therefore blocking calls, blocking until the rendez-vous happens, both returning a fully connected full-duplex channel.

When connecting, we may want to distinguish between two cases:

- there is no pending accept that corresponds yet
- there is no broker with the given name

When the named broker does not exist, the connect returns null. To be retried later if the broker is known to be created at some later time.

If the remote broker is found, the connect blocks until there is a matching accept so that a connected channel can be constructed and returned.

Note: we could consider introducing a timeout here, limiting the wait for the rendez-vous to happen.

Writing

Signature:

```
write(byte[] bytes, int offset, int length) int
```

When writing, the given byte array contains the bytes to write from the given offset and for the given length. The range [offset,offset+length[must be within the array boundaries, without wrapping around at either ends. If it is not the case, an illegal-argument exception is thrown.

The method "write" returns the number of bytes actually written, that number may not be zero or negative. If zero would be return, the write operation blocks instead until it can make some progress.

Nota Bene: a channel is a stream, so although the write operation does take a range of bytes to write from an array of bytes, the semantics is one that writes one byte at a time in the stream.

The method "write" blocks if there is no room to write any byte, but does not have to write all the given bytes. The rationale is to avoid spinning when an application tries to send a certain number of bytes and the stream can make no progress. Here is an example:

```
void send(byte[] bytes) {
    int remaining = bytes.length;
    int offset = 0;
    while (remaining!=0 && !channel.disconnected()) {
        int n = channel.write(bytes,offset,remaining);
        offset += n;
        remaining -= n;
    }
}
```

```
}  
}
```

Invoking a write operation on a disconnected channel simply drops the bytes. The rationale is that it is simpler since the case of dropping bytes is unavoidable in an asynchronous world. Indeed, there will always be a time-window between the time the remote end of a channel is disconnected and the time that fact is known locally. During that time window, some bytes may be written, sent through, and will effectively be lost because the other side will disconnect before they are received and read. Of course, network failures and task failures are also imposing that same behavior.

Nota Bene: thus, a write operation that is blocked must unblock when the channel becomes disconnected, dropping bytes if necessary.

Nota Bene: there is no error case... like returning -1 or throwing an exception. The rationale is that an error is considered equivalent to a disconnect. We may want to add a cause to a disconnection, for information.

Reading

Signature:

```
read(byte[] bytes, int offset, int length) int
```

When reading, the given byte array will be used to store the bytes read from the channel, starting at the given offset. The given length provides the maximum number of bytes to read. The range [offset, offset+length[must be within the array boundaries, without wrapping around at either ends. If this is not the case, an illegal-argument exception is thrown.

The method "read" will return the number of bytes actually read, that number may not be negative.

- A read operation may only return zero if that read operation has no bytes available and the channel is disconnected. This means that a blocked read will unblock and return 0.
- With a connected channel, a read operation will return the number of bytes read and copied in the given array of bytes. That number will always be less or equal to the given length. If there are no available bytes at all, the method "read" blocks until some bytes become available.

The rationale is that a loop trying to read a given length, looping over until all the needed bytes are read will not induce an active polling. Here is an example:

```
void receive(byte[] bytes) {  
    int remaining = bytes.length;  
    int offset = 0;  
    while (remaining != 0 && !channel.disconnected()) {  
        int n = channel.read(bytes, offset, remaining);  
        offset += n;  
        remaining -= n;  
    }  
}
```

```
}  
}
```

Nota Bene: a remotely disconnected channel will allow read operations locally until all the bytes that have been remotely written are read. During that time, the channel is not considered as disconnected locally, so the method `disconnected()` `boolean` will return `false`.

Nota Bene: there is no error case... like returning -1 or throwing an exception. The rationale is that an error is considered equivalent to a disconnect. We may want to add a cause to a disconnection, for information.

Disconnecting

A channel can be disconnected at any time, from either side. So this requires an asynchronous protocol to disconnect a channel. The effect of disconnecting a channel must be specified for both ends, the one that called the method "disconnect" as well as the other end.

Nota Bene: both ends may call the method "disconnect" concurrently and the protocol to disconnect the channel **must** still work.

In the following, we will talk about the local side versus remote side, the local side being the end point where the method "disconnect" has been called.

It is always legal to:

- invoke the method "disconnected" to check the status of the channel.
- invoke the method "disconnect" multiple times, only the first invocation does something, others are simply ignored.

It is important to state that since we have not asserted a strict ownership model between tasks and channels, it is possible that a channel be disconnected by one task while some operations, called on other tasks, are blocked locally. These blocked operations must be interrupted, when appropriate:

- a block read will return 0.
- a block write will drop the remaining bytes.

The main issue when disconnecting a channel is that there may be bytes still in transit while a channel is half-connected. Bytes in transit were written by the local side before it disconnected the channel. The channel must guarantee that these bytes can be read by the remote side before the channel appears to be disconnected to that remote side. This behavior is mandatory since it is likely that a communication will end by one side writing some last bytes and then disconnecting.

Brokers and Multi-tasking

Let's discuss now the relationship between tasks, brokers, and channels. Since both the operations 'connect' and 'accept' are blocking operations, we can state the following:

- A task cannot connect to the same name and port concurrently, but multiple tasks can.
- Only one task may accept on a given port on a given broker.

- Different tasks on different brokers may accept on the same port number.
- Multiple tasks may accept on different ports on the same broker.

We know that each task is related to a broker, by its constructor. But a broker may be aliased across tasks, running in the same Java Runtime Environment (JRE), meaning that a broker can be used by multiple tasks. In other words, brokers may be shared between tasks, so **brokers must be thread-safe**.

Channels only need to be partially thread-safe:

- Multiple concurrent invocations of the method 'write' on one channel by multiple tasks is not supported.
- Multiple concurrent invocations of the method 'read' on one channel by multiple tasks is not supported.
- Two tasks may invoke concurrently the method 'read' and 'write' on the same channel.

Therefore, tasks that cooperate on one end point of a channel, invoking the methods 'read' and 'write' concurrently, must synchronize themselves according to the above rules.

Classical cooperation patterns are the following:

- one task owns one end point of a channel, doing both the read and write operations.
- two tasks share one end point of a channel, one task reading and the other writing.
- one task does the reading and other tasks are synchronized to do writes.